

Test Project Outline – Module D – GIFTS Training Manager

Competition time

3 hours

Introduction

Build a **standalone frontend prototype** for **GIFTS** (Global Institute for Transferring Skills), an affiliated organization of the Human Resources Development Service of Korea (HRDKorea) that manages WorldSkills Korea, to coordinate preparation for web technologies training. Training activities focus on project tasks from **MITS** (Marketable IT Skills, <https://skillsit.eu>), a large collection of standardized test projects adapted from national and international skills competitions. The full product will include a REST API backend; this module is **client-side only**: load JSON on startup, persist changes in the browser, and support JSON export/import.

General Description of Project and Tasks

The **GIFTS Training Manager** helps training coordinators and competitors plan and monitor a **120-day** WorldSkills web technologies training programme. Using the application, they can:

- view and edit the training schedule on a **monthly calendar** — add, move, or remove events per day through a day-details modal;
- track **project tasks** and **evaluations** from the MITS catalog, plus custom **other** events (meetings, mental training, and similar);
- follow task progress on a **weekly Kanban board** (ToDo, InProgress, Done);
- review **statistics** — completed days, planned vs completed hours, scheduling gaps, and day-coverage warnings.

The application loads an initial schedule and the MITS task catalog from JSON files, **persists all changes in the browser**, and supports **export and import** of the schedule so plans can be backed up or shared. Built-in **business rules** enforce daily event limits, hour limits, and how often each MITS task may be scheduled.

Technology

HTML, CSS, JavaScript (framework permitted).

Data files and assets

- `assets/data/training.json` (initial schedule)
- `assets/data/collection.json` (MITS task catalog)
- SVG icons in `assets/svg/`
- GIFTS logo (`assets/images/gifts-logo.png`)
- design files (`assets/design/`).

Data model

Initial `training.json` has **exactly 120** entries in `days`.

```
{
  "version": 1,
  "trainingPeriod": { "start": "2026-02-01", "end": "2026-08-31" },
  "days": [
    {
      "date": "2026-02-03",
      "events": [
        {
          "type": "task",
          "name": "ES2023 S17 - Module D",
          "durationHours": 4,
          "kanbanStatus": "todo"
        },
        {
          "type": "evaluation",
          "name": "ES2023 S17 - Module D",
          "durationHours": 2,
          "kanbanStatus": "todo"
        }
      ]
    }
  ]
}
```

Day:

- `date` — ISO `YYYY-MM-DD` within the training period
- `events` — array of 1–3 event objects

Event:

- `type` — `task`, `evaluation`, or `other`
- `name` — must match a `collection.json` entry for `task` and `evaluation`; free text for `other`
- `durationHours` — whole hours from 1 to 9
- `kanbanStatus` — `todo`, `inProgress`, or `done`; required for `task` and `evaluation` only
- `description` — required for `other` events only

Requirements

Initial load and persistence

Store the training schedule in the browser so users do not lose work when they refresh the page or close and reopen the browser.

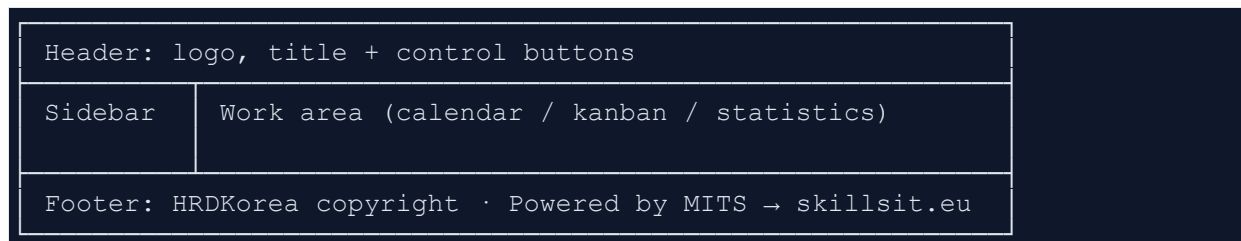
On the first visit, or whenever browser storage is empty, load `assets/data/training.json` (initial schedule) and `assets/data/collection.json` (MITS task catalog) from `assets/data/`. The catalog is reference data only; user edits affect the schedule, not the catalog file.

Persist every change to the training schedule — adding, editing, moving, or deleting events, and updating Kanban status — so the stored state always reflects the user's latest work.

Application layout

Implement single-page layout with four regions: **header**, **sidebar**, **work area**, and **footer**. Main features must be usable at least on desktop resolution.

Design reference images in `assets/design/` may be used as a guide, but **pixel-perfect reproduction is not required**. You may apply your own visual design as long as it meets the functional and layout requirements in this document.



Header

Element	Description
Application logo & title	GIFTS logo with Training Manager title
Theme toggle	Light / dark mode (persists after reload)
Fullscreen	Toggle fullscreen on / off
Export	Download the current training schedule as a JSON file (see Export and import)
Import	Load a previously exported JSON file and replace the stored schedule (see Export and import)

Sidebar

Vertical navigation menu; the **active view is visually highlighted**.

Menu item	View
Month (Calendar)	Monthly calendar grid

Menu item	View
Kanban	Weekly Kanban board
Statistics	Summary reports and warnings

Work area

Main content beside the sidebar — content changes with the selected view:

- **Calendar:** monthly grid with previous / next month navigation
- **Kanban:** three columns for the selected week with week navigation
- **Statistics:** metrics and day-coverage warnings

Use provided SVG icons from `assets/svg/` where appropriate (calendar, kanban, export, import, theme, navigation, and similar).

Footer

Element	Description
Copyright	HRDKorea copyright notice
Attribution	Powered by MITS — link to https://skillsit.eu

App color palette

Use the GIFTS logo palette as the **app color palette**:

Color	Hex	Suggested use
Blue	#4462b1	Primary UI accents, <code>task</code> badges/chips
Green	#59a149	<code>evaluation</code> badges/chips
Purple	#6f2a90	<code>other</code> badges/chips, secondary accents

Apply these colors consistently across the interface (header, sidebar, buttons, type indicators). Both light and dark modes should preserve recognizable GIFTS brand colors.

Theme toggle

- Add a **light/dark theme** toggle button in the header. The selected theme must **persist after reload**.

Fullscreen

- Add a **fullscreen** toggle button in the header to enter and exit fullscreen mode.

Monthly calendar

On startup, the calendar shows the **current month**. Navigate months within the training period. The full month grid may be shown; days **outside** the training period appear **dimmed**. Cells show a **summary** only — all viewing and editing happens in the **modal**.

Day status (visual only)

Status	Appearance
Past	Dimmer / greyer background or text
Today	Highlighted border
Future	Normal appearance

Day cell content

On days **with events**, each cell shows:

Element	Display
Sequence #_n	1-based index in ascending date order in days ; # prefix (e.g. #42); top-right corner; recalculated after add / delete / reschedule; not shown if the day has no events

Element	Display
Name	If all events share the same <code>name</code> , or there is only one event: show <code>name</code> , max 1 lines with ... overflow. If names differ: Multiple different events
Total hours	Sum of <code>durationHours</code> (e.g. <code>6h</code>)
Type badges	T task, E evaluation, O other — only types present; color-coded per the GIFTS color palette (blue / green / purple)

Examples: task + evaluation same name → #42, ES2023 S17 - Module D, 6h, TE; mixed names → Multiple different events with hours and badges.

On **empty days** (no events): no summary, only the day number.

Day details modal

Opens on cell click.

Display: date, sequence number, and the full event list for that day. For each event show `type`, `name`, and `durationHours`. For `task` and `evaluation` events, also show `displayName` from `collection.json` and `kanbanStatus`. For `other` events, show `description`.

Editing:

- **Delete:** click the trash icon on an event.

- **Reschedule:** each event shows a date field (inline editing or date picker) and a **Move** button. Click **Move** to apply the new date.
- **Add:** form within the modal.
 1. Select event `type`.
 2. **task or evaluation:** show a MITS task picker listing only `collection.json` entries with **fewer than 2** assignments of the selected type; each option displays the entry's `name`. Default duration: `estTime` from `collection.json` for task; **2 hours** for evaluation (editable).
 3. **other:** require `name`, `description`, and `duration` (whole hours, 1–9).
- Enforce per-day limits (**1–3** events, **≤ 9** hours total). `durationHours` must be a whole hour between **1** and **9**.
- Invalid changes are rejected with an error message in the modal.
- **Save** and **Cancel** buttons apply or discard edits.

Enforce all rules below on every relevant operation. Show user-friendly validation messages (e.g. "This day has reached the 9-hour limit", "Duration must be a whole hour between 1 and 9", "122 days already have events — cannot schedule to a new day") when a change is invalid.

1. At most **122 days** may have events.
2. Only days within the defined **training period** may have events.
3. Event types must be `task`, `evaluation`, or `other`.
4. Each day has at most **3 events**.
5. Each event must have **1–9 whole hours** duration (0 and fractional hours are rejected).
6. The **daily total** of all events on that day must not exceed **9 hours**.
7. Tasks and evaluations must be chosen **only** from `collection.json`.
8. Each MITS project task from `collection.json` may be scheduled **at most twice** as a `task` event and **at most twice** as an `evaluation` event. (Each MITS project task from `collection.json` has a `uniqueName` property.)

Close via **Esc**, click outside, or a **Close** button — the calendar refreshes on close.

Kanban board

Tasks and evaluations only — `other` events do **not** appear on the board.

One week (Monday–Sunday) at a time; **previous / next week** buttons; on startup select the current week or the week containing today.

Column	Maps to <code>kanbanStatus</code>
ToDo	<code>todo</code>
InProgress	<code>inProgress</code>

Column	Maps to <code>kanbanStatus</code>
Done	<code>done</code>

Drag & drop between columns updates status and persists (including after page refresh).

Kanban card layout

Each card shows:

Element	Display
Type chip	Rounded badge at the top left: TASK — GIFTS blue background, white uppercase text; EVALUATION — GIFTS green background, white uppercase text
Title	<code>displayName</code> from <code>collection.json</code> (bold), resolved at runtime via <code>name</code>
Name	Collection entry <code>name</code>
Details	Associated day date and duration, e.g. <code>2026-06-17 · 3h</code>

Statistics

The statistics view must display the following info:

- number of completed training days (days where all assigned `task` and `evaluation` events are marked done);
- total planned hours vs. completed hours (only `task` and `evaluation` events must count);
- MITS scheduling status: list only `collection.json` entries with fewer than 2 `task` assignments or fewer than 2 `evaluation` assignments, showing the current count for each;
- Day-coverage warnings (if applicable):

Condition	Message (example)
Fewer than 120 days have events	"Warning: only {n}/120 training days have active events."
More than 122 days have events	"Warning: the number of days ({n}) exceeds the allowed maximum (122)."

Export and import

Header buttons let users back up and restore the training schedule.

Export

- Triggered from the **Export** button in the header.
- Downloads the **current persisted schedule** as a JSON file (browser file save).
- JSON uses the same structure as the provided `training.json`.
- Exported `days` contain **only days with events**.
- Output is **human-readable** (formatted / pretty-printed).

Import

- Triggered from the **Import** button in the header.
- Opens a file picker for a JSON file previously exported from the app (or compatible with the data model).
- Shows a **confirmation** dialog before overwriting the stored schedule; canceling leaves the current data unchanged.
- On confirm: replace persisted state with the imported data, enforce business rules, and **refresh all views** (calendar, Kanban, statistics).
- Reject invalid files with a clear error message (e.g. malformed JSON, missing required fields, or data that violates business rules).

Assessment

Evaluation in **Google Chrome** — business rules, persistence, calendar, Kanban, statistics, and source code.

Mark distribution

Total: **21 points**. See `marking/marking-scheme.json` for aspect-level detail.

WSOS SECTION	Description	Points
1	Work organization and self-management	1.5

WSOS SECTION	Description	Points
2	Communication and interpersonal skills	1.5
3	Design Implementation	4.75
4	Front-End Development	13.25
Total		21